Oppgave 1 (25% ~1 time) – ORM/JPA

Vi skal jobbe litt med en database for et bibliotek. Databasen skal ha oversikt over **bøker**, **medlemmer** (en person registrerer seg som medlem for å kunne låne bøker) og **utlån**.

Vi tenker oss følgende tabeller. Det er tatt med eksempeldata for å illustrere mulig innhold.

bok

id (PK) **Surrogatnøkkel**	tittel	forfatter	hylle
1	'Wuthering heights'	'Emily Brontë'	'A4.3'
2	'1984'	'George Orwell'	'A1.1'
3	'Lord of the Rings'	'J. R. R. Tolkien'	'A2.5'
…	…	…	…

medlem

nr (PK) **Surrogatnøkkel**	fornavn	etternavn	telefon
10001	'Atle'	'Patle'	'98765432'
10002	'Bob'	'Byggmester'	'48484848'
10003	'Per'	'Viskelær'	'12345678'
…	…	…	…

utlaan

id (PK) **Surrogat** **…**	bokid (FK)	medlemnr (FK)	utlaandato	forfalldato	returdato
1	2	10003	'2024-01-31'	'2024-02-29'	'2024-02-15'
2	2	10001	'2024-02-16'	'2024-03-16'	**NULL**
3	3	10001	'2024-02-16'	'2024-03-16'	**NULL**
…	…	…	…	…	…

Returdato **NULL** betyr at boken ikke er returnert.

I **utlaan**-tabellen ser vi f.eks. at bok med id=2 (1984 av George Orwell) er først lånt ut til medlem nr 10003 (Per Viskelær) og returnert 15. februar, og deretter lånt ut til medlem nr 10001 (Atle Patle) og enda ikke returnert.

Utlaan-tabellen ivaretar altså historikken for alle utlån.

Vi ønsker å jobbe med denne databasen i et Java-program.

a) (9% ~ 20 min) Skriv Java-klassene for entitetstypene **Bok, Medlem** og **Utlaan** som tilsvarer tabellene i eksempelet vist over. Du trenger ikke å skrive metoder, kun instans-/objektvariabler og JPA-annoteringer. Husk de nødvendige annoteringer for å angi forholdet mellom entitetene:
   - Vi ønsker å ha **toveis navigering** i Java mellom **Medlem** og **Utlaan**, slik at et medlem har en liste over hvilke utlån han/hun har foretatt.
   - Vi ønsker **IKKE** å ha toveis navigering i Java mellom **Bok** og **Utlaan**. Hvis vi ønsker en oversikt over utlånene til en bok, må det lages en egen spørring for dette.

Løsningsforslag:

```
1   @Entity
    public class Bok {

1       @Id
1       @GeneratedValue(strategy = GenerationType.IDENTITY)
        private int id;

        private String tittel;
        private String forfatter;
        private String hylle;

-2      //Trekk hvis denne eller tilsvarende er med!
        @OneToMany(mappedBy = "bok", fetch = FetchType.EAGER)
        private List<Utlaan> utlaan;

        ...
    }

    @Entity
    public class Medlem {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private int nr;

        private String fornavn;
        private String etternavn;
        private String telefon;

1       @OneToMany(mappedBy = "medlem",
1                  fetch = FetchType.EAGER)
1       private List<Utlaan> utlaan;

        ...
    }
```

```
    @Entity
    public class Utlaan {

        @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
        private int id;

1       @ManyToOne @JoinColumn(name = "bokid")
1       private Bok bok;

0.5     @ManyToOne @JoinColumn(name = "medlemnr")
0.5     private Medlem medlem;

1       private LocalDate utlaandato;
        private LocalDate forfalldato;
        private LocalDate returdato;

        ...
    }
```

Du kan i de videre oppgavene anta at entitetsklassene inneholder de nødvendige konstruktører, gettere og settere, etc. du trenger i løsningene dine.

Vi antar at vi har en hjelpeklasse **BibliotekDAO** for databaseoperasjoner. Du skal lage et par metoder i denne. Du kan anta at en EntityManagerFactory kalt emf er opprettet og tilgjengelig for bruk i metodene.

b) (4% ~ 10 min) Skriv en metode **Medlem finnMedlemMedNr(int medlemnr)** i BibliotekDAO som henter ut medlemmet med gitt medlemnr. Hvis ingen medlemmer med dette nummeret finnes, skal det returneres null.

```
Løsningsforslag:

    public Medlem finnMedlemMedNr(int medlemnr) {

2       EntityManager em = emf.createEntityManager();

        try {
2+2+2       return em.find(Medlem.class, medlemnr); // null om ikke funnet

        } finally {
2           em.close();
        }
    }
```

```
PS! Hvis du bruker try-with-resources (EntityManager er AutoCloseable, og vil da
automatisk lukkes når try-blokken er ferdig) kan løsningen forenkles slik:

    public Medlem finnMedlemMedNr(int medlemnr) {

4       try(EntityManager em = emf.createEntityManager()) {
6           return em.find(Medlem.class, medlemnr);
        }
    }
```

```
Tilsvarende gjelder også c) og d).
```

c) (4% ~ 10 min) Skriv en metode **List<Bok> finnBokerPaaForfatter(String forfatter)** i BibliotekDAO som henter ut alle bøkene til en gitt forfatter.

```
Løsningsforslag:

    public List<Bok> finnBokerPaaForfatter(String forfatter) {

        String queryString
3                   = "SELECT b FROM Bok b WHERE b.forfatter = :forfatter";

        EntityManager em = emf.createEntityManager();
        try {
2            TypedQuery<Bok> query = em.createQuery(queryString, Bok.class);
2            query.setParameter("forfatter", forfatter);
3            return query.getResultList();
        } finally {
em.close();
        }
    }

PS! Metodekall til query kan også «kjedes» (method chaining), slik at try-blokken
kan skrives som én kodelinje:

        try {
return em.createQuery(queryString, Bok.class)
                    .setParameter("forfatter", forfatter)
                    .getResultList();
        } finally {
             ...
```

d) (8% ~ 20 min) Skriv en metode **boolean leverTilbakeBok(int bokid)** i BibliotekDAO som markerer gitt bok som tilbakelevert med returdato lik dagens dato. Hvis alt går bra skal metoden markere boken som tilbakelevert, og returnere true. Du skal gjøre feilhåndtering for tilfellene der det **ikke finnes** et ikke-returnert utlån for denne boken, og der det **finnes flere** ikke-returnerte utlån for denne boken. Ved feil skal det ikke gjøres noen oppdateringer, og returneres false.

Litt hjelp til løsningen:

- **query.getSingleResult()** kaster hhv. NoResultException og NonUniqueResultException ved ingen eller flere treff.
- **query.getResultList()** inneholder 0 hhv. flere objekter ved ingen eller flere treff.
- Dagens dato fås ved å bruke **LocalDate.now()**.

```
Løsningsforslag:

Et par viktige ting å merke seg:
        - Informasjon om utlån ligger i utlaan-tabellen / Utlaan-klassen,
          ikke i bok-tabellen!
        - bokid er ikke primærnøkkel i utlaan / Utlaan. Vi kan derfor ikke
```

```
          bruke find(), men med gjøre en spørring!
        - Etter setting av returdato for utlånet skal det ikke gjøres en
          persist(), da dette oppretter en ny rad i tabellen! Det er heller
          ikke nødvendig med merge() siden utlaan-objektet allerede er managed.

    public boolean leverTilbakeBok(int bokid) {

        EntityManager em = emf.createEntityManager();
        EntityTransaction tx = em.getTransaction();

2       String queryString = """
SELECT u FROM Utlaan u
WHERE u.bok.id = :bokid
AND u.returdato = null """;

        try {
1           tx.begin();

1           TypedQuery<Utlaan> query = em.createQuery(queryString, Utlaan.class);
1           query.setParameter("bokid", bokid);

            //Alt.1 – getSingleResult(). Feil håndteres i catch-blokk.
1           Utlaan utlaan = query.getSingleResult();

2           utlaan.setReturdato(LocalDate.now());

1           tx.commit();
            return true; // alt ok

1       } catch (NoResultException | NonUniqueResultException e) {
            tx.rollback();
            return false; //noe gikk galt

        } finally {
            em.close();
        }
    }
```

```
//Alternativ løsning (som noen har prøvd på)
      JPQL har også mulighet til å gjøre UPDATE, via:

«UPDATE Utlaan u SET u.returdato = :returdato WHERE u.bok.id = :bokid ...»
             +
query.executeUpdate()

, men dette er ikke gjennomgått i kurset
, og er egentlig mer beregnet på bulk-oppdatering !
```

Oppgave 2 (25% ~1 time) – modellering/normalisering

Problembeskrivelse

Et firma selger varer på Internett. De har behov for en database for å håndtere logistikken rundt dette (vi ser her bare på selve salget – ikke lager-beholdning/-føring). Når en kunde ønsker å bestille noe, blir det opprettet en ordre (bestilling). Denne har en "liste" av varer som blir bestilt. For hver ordre er det en egen ordre-id. En kunde kan opprette så mange ordrer han/hun ønsker (men det er alltid bare en kunde som er mottaker av en ordre). Det er mange ulike varer til salgs. For en vare som blir solgt er det nødvendig å vite navn (på varen), antall (kunden har bestilt), pris og eventuell rabattkode. Hver kunde blir tildelt en unik kunde-id, og det er i tillegg behov for å vite navn, epost, adresse, postnr og poststed.

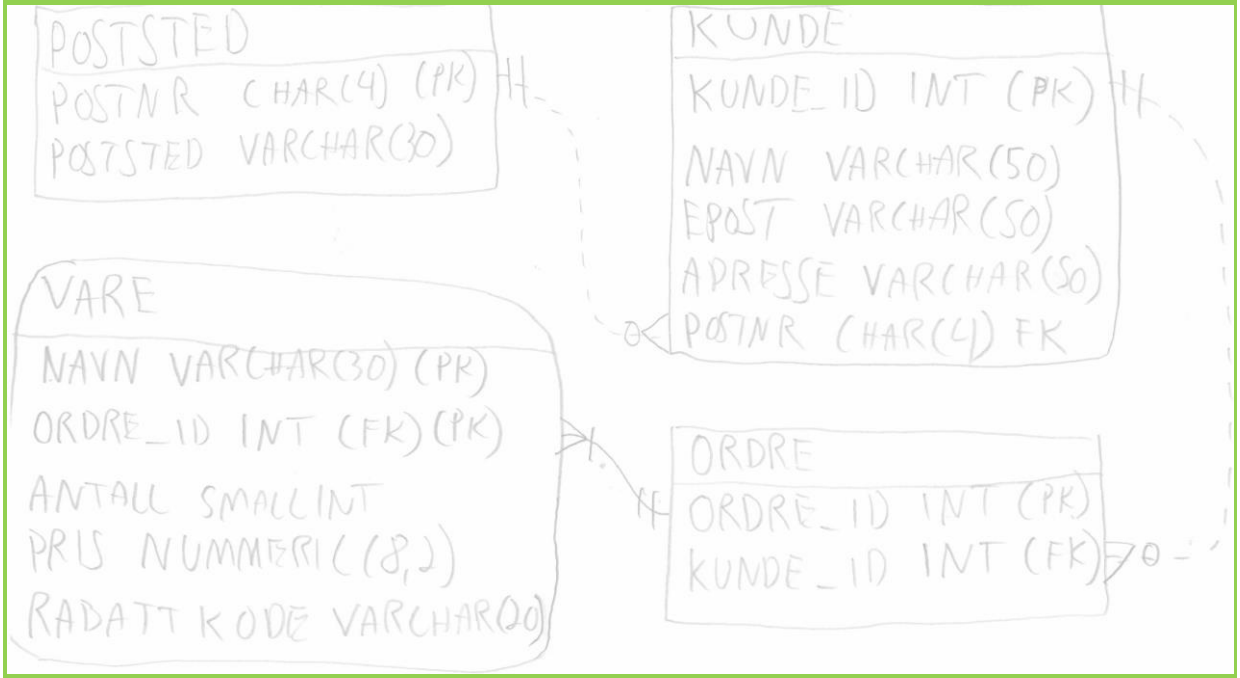
Oppgave

- Lag (tegn) en logisk (fullstendig) ER-modell for denne databasen i en av notasjonene som har vært brukt i kurset.
- Vis at tabellene for databasen tilfredsstiller minst 3. normalform.

Huskeliste for hva besvarelsen skal (minst) inneholde

- Oppgaven kan bli besvart helhetlig (istedenfor punktvis) i den grad du mener dette er enklere. Men alle punktene skal besvares.
- Bestem hva som er hensiktsmessige tabeller for denne databasen.
- Primærnøkler (som du mener er mest hensiktsmessig)
- Fremmednøkler
- Datatyper
- Min/maks kardinalitet
- Sterke/svake entitetstyper og eksistensavhengighet/uavhengighet (kråkefot-notasjon) eller type aggregering (UML-notasjon)
- Forklaring på at databasen tilfredsstiller hver av 1., 2., og 3. normalform.
- **Forklaring for de valgene som er tatt. Det blir lagt vekt på både praktisk utførelse og teoretisk forståelse.**
- **Besvarelsen skal være "konsistent", dvs. besvarelsen være for den "samme" databasen, og de valgene man har tatt i en del av besvarelsen skal ikke "motsies" av de valgene man har tatt i en annen del av besvarelsen.**
- (Dersom du har problemer med deler av oppgaven, prøv å gjøre det du kan. Du kan for eksempel lage en "forenklet" modell, og forklare hvorfor du mener den tilfredsstiller normalformene.)

Løsningsforslag



ER-modellen ovenfor er trolig den som er nærmest det som står i" kravspesifikasjonen". Men også andre løsninger kan bli godtatt hvis de kan løse oppgaven på en tilfredsstillende måte. Det er viktig å merke seg at kravspesifikasjonen ikke krever i tabell for varelager. Det er tilstrekkelig med en tabell for varer som er solgt. Slik denne er brukt i løsningsforslaget, svarer den mer til en tabell Ordrelinje som viser salg av en bestemt vare på en gitt ordre. (Man kunne i prinsippet tenke seg at kolonnen Navn er en fremmednøkkel til en varelager-tabell som ikke er med i oppgaven.) Det er en funksjonell avhengighet mellom postnummer og poststed, og det er derfor forventet at Poststed må legges ut som en egen tabell for at løsningen skal være normalisert.

Vi forutsetter at en Kunde må ha et poststed (vi må kunne sende varer til kunden), og at en kunde ikke kan ha mer enn et poststed. Et poststed må derimot ikke ha noen kunder (vi kan ha en liste over alle norske poststeder), og det kan selvsagt bo mange kunder på samme poststed. Det er naturlig at en kunde kan opprette mange ordrer. Det kan diskuteres om en kunde må ha minst en ordre, man trolig vil det være mest naturlig at en kunde kan opprettes i systemet før denne faktisk oppretter ordrer. En ordre kan derimot bare ha en kunde. Det er denne kunden som mottar ordren (og formodentlig må betale for den). Det re heller ikke mulig å ha ordrer uten kunder – hvor skulle vi sende varer/regning? En Ordre kan bestå av mange Varer. Det er vanlig å kunne bestille flere varer på samme Ordre. Man kan diskutere om det bør være mulig med Ordrer som ikke har Varer. Kanskje kan man først opprette en Ordre, og så legge til Varer? Kanskje – men det virker mer naturlig at en ordre blir opprettet av systemet når en kunde har valgt en eller flere varer og ønsker å bestille disse. Slik tabellen Vare er brukt i dette løsningsforslaget, så gir det bare mening at den må ha en Ordre. Husk – en Vare her er ikke en vare på et Varelager – men en Vare kunden har bestilt på en gitt Ordre. Som sagt minner den her mer om en ordrelinje, med detaljer for selve salget, og ikke de egenskaper varen i seg selv har. Således kan en Vare heller ikke ha flere enn en Ordre – Varen er knyttet til en bestemt Ordre. Vi ser for øvrig at tabellen Vare har en sammensatt primærnøkkel av Navn og Ordre_id. Vi merker oss også at tabellen Vare dermed er en svak entitetstype siden den arver del av primærnøkkel fra Ordre, og følgelig også eksistensavhengig av Ordre. Avrundede hjørner og heltrukken linje indikerer dette.

For Poststed er Postnr en opplagt primærnøkkel. For Kunde har vi et løpenummer Kunde_id som primærnøkkel, og ordre har på samme måte et løpenummer Ordre_id som primærnøkkel. For Vare er primærnøkkelen som sagt sammensatt av Navn (kunne også vært fremmednøkkel til en "hypotetisk" varelager-tabell) og Ordre_id.

For Postnr er det mest naturlig å bruke char(4), siden postnumre kan starte med 0. For de fleste tekstfeltene vil det være naturlig å bruke varchar(...) med en lengde som passer til de dataene de skal lagre. En viss variasjon vil be akseptert her. For Kunde_id og Ordre_id er trolig int mest fornuftig. Trolig ville man bruke automumerering med datatypen serial, og denne er egentlig av typen int. Men en variant med char(...) er også akseptabel. For Antall passer trolig int best, men hvis det kan være aktuelt å selge ting per vekt, lengde eller volum, så kan også float være en mulighet. Som regel vil numeric være mindre aktuelt, siden denne først og fremst brukes når det er et stort poeng at en verdi skal kunne lagres nøyaktig i 10-tallsystem, og brukes hovedsakelig for monetære (penger) verdier. Dette er derimot tilfelle for pris.

Vi ser at ingen av tabellene inneholder kolonner som ikke er "atomaære", dvs., består av "lister". Databasen tilfredsstiller derfor 1. normalform. Vi ser deretter etter "partielle avhengigheter". Dette er tilfeller hvor kolonner avhenger bare av del av (sammensatt) primærnøkkel. De tabellene som kun har en enkelt (ikke sammensatt) primærnøkkel vil dermed automatisk tilfredsstille dette. Eneste tabellen med sammensatt primærnøkkel er Vare. Vi ser at ingen av kolonnene er avhengig av bare en del av primærnøkkel. Vi kunne innvende at Pris bare avhenger av Navn, men her er det viktig å huske a salgsprisen for en vare kan endre seg over tid, og at pris her reflekterer den prisen kunden har betalt for varen på akkurat denne Ordren – ikke den generelle utsalgsprisen. Databasen er dermed også på andre normalform. Vi må deretter se etter transitive avhengigheter. Dette er kolonner som avhenger av en annen kolonne enn primærnøkkel Et mulig brudd kunne være Postnr og Poststed., siden Poststed som sagt avhenger av Posnr. Men dette har vi allerede ordnet opp i, og følgelig ikke et problem. Heller ikke noen av de andre tabellene synes å ha kolonner som avhenger av en kolonne som ikke er primærnøkkel. Adresse er ment å inneholde gatenavn og husnummer. Man kan innvende at det kan være avhengigheter her. Denne problemstillingen er imidlertid for omfattende til å gå inn på her. Vi konkluderer derfor med at databasen tilfredtiller til og med tredje normalform.

Oppgave 3 (25% ~1 time) – SQL

Problembeskrivelse

Denne oppgaven er løst basert på problembeskrivelsen i Oppgave 2 – modellering/normalisering. Du kan anta at en mulig, "redusert" løsning inneholder tabellene:

- Kunde(kunde_id, navn, epost)
- Ordre(ordre_id, kunde_id*)

Disse tabellene skal være utgangspunkt for besvarelsene i denne oppgaven. Du kan derfor løse denne oppgaven selv om du ikke fikk til Oppgave 2.

Oppgave

a) Skriv fullstendig SQL-kode for å opprette tabellen Kunde. Du kan anta at tabellen Ordre eksisterer.
b) Skriv SQL-kode for å legge inn en kunde (med selvvalgte data)
c) Skriv SQL-kode for å legge til en kolonne Epost til tabellen Kunde
d) Skriv SQL-kode for å opprette en indeks på kolonnen Epost
e) Skriv SQL-kode for å vise all info om alle ordrer. Sorter synkende på Kunde_id
f) Modifiser svaret på e) slik at den for hver ordre også viser all info om tilhørende kunde
g) Skriv spørringen din fra spørsmål f) som relasjonsalgebra
h) Skriv SQL-kode for å vise all info om alle kunder. For hver kunde skal også antall ordrer denne kunden har vises.
i) Vil spørringen din fra spørsmål h) også inkludere kunder som ikke har noen ordrer? Hva avgjør dette? Modifiser svaret ditt for å gjøre det "omvendt"
j) Lag et "enkelt" overslag over hvor mye plass en database med 100 kunder og 1000 ordrer vil bruke (bare med disse to tabellene som blir brukt i denne oppgaven)

Huskeliste for hva besvarelsen skal (minst) inneholde

- Primærnøkler (som du mener er mest hensiktsmessig)
- Fremmednøkler
- Datatyper
- Null/not null
- Eventuelle fremmednøkler og on update/delete krav.
- **Forklaring for de valgene som er tatt. Det blir lagt vekt på både praktisk utførelse og teoretisk forståelse.**
- **Besvarelsen skal være "konsistent", dvs. besvarelsen være for den "samme" databasen, og de valgene man har tatt i en del av besvarelsen skal ikke "motsies" av de valgene man har tatt i en annen del av besvarelsen.**
- (Dersom du har problemer med deler av oppgaven, prøv å gjøre det du kan. Du kan for eksempel lage en "forenklet" modell, og forklare hvorfor du mener den tilfredsstiller normalformene.)

Løsningsforslag

a) Create table kunde(
kunde_id serial primary key,
navn varchar(50) not null,
epost varchar(50) null
);
Denne tabellen har ingen fremmednøkler, så det skal ikke være bruk av on update/delete. Trolig vil det være mest naturlig å kreve at kunde har navn (not null), mens man kanskje bør ta høyde for at det kan være kuner som ikke har epost. Det er kom for å bruke andre valg av datatyper og lignende, men de bør være "fornuftige".

b) Insert into kunde (navn, epost) values ('Kari Nordmann', 'kari@nordmann.no');
Siden vi har valgt autonummerering av kunde_id, bør vi ikke angi kundenummer har (dersom man brukte int eller char må man selvsagt det). Vi angir derfor de kolonnene vi har behov for å sette, og tilhørende verdier.

c) Alter table kunde add column epost varchar(50) null;
Her var det i oppgaveteksten egentlig tenkt å bruke telefonnummer, men det ble ved en feil brukt epost istedenfor, selv om tabellen allerede har denne kolonnen. Det er selvsagt beklagelig, men vi håper at dette ikke har medført problemer. Som tidligere nevnt, så er det trolig mest fornuftig å tillate null-verdier her (men annet kan argumenteres for).

d) Create index epost_index on kunde(epost);
Her har vi valgt kolonnen epost i tabellen Kunde, men kunne også vært en annen kolonne.

e) Select *
from ordre
order by kunde_id desc;
Dette var egentlig ment å være en veldig enkel spørring, som bare involverer en tabell. Standard sortering er stigende, så man må legge til desc.

f) Select *
from ordre, kunde
where ordre.kunde_id=kunde.kunde_id
order by order.kunde_id desc;
Her har vi utvidet forrige spørring med en enkel likekobling for å få med info om kunden. Bruk av inner join, left outer join eller right outer join blir også generelt sett akseptert.

g) ordre $\bowtie_{\text{ordre.kunde_idr=kunde.kunde_id}}$kunde
Det er rom for å bruke ulike tegn og notasjon for å få frem dette, men det bør i all hovedsak minne om dette.

h) Select kunde.*, count(ordre.kunde_id)
from kunde inner join ordre on kunde.kunde_id=ordre.kunde_id
group by kunde.kunde_id;

Har har vi brukt en inner join for å koble sammen tabellene. Siden oppgaven ikke setter noen spesielle krav til koblingen, kunne man også brukt en likekobling eller en varient av left/right outer join (men ikke full outer joun) dersom de blir brukt riktg. Det er viktig at count(...) og group by blir brukt riktig.

i) Select kunde.*, count(ordre.kunde_id)
from kunde left outer join ordre on kunde.kunde_id=ordre.kunde_id
group by kunde.kunde_id;
Her skal man først vurdere om svaret ditt på forrige oppgave vil ta med kunder som ikke har noen ordrer. Dersom man i forrige oppgave brukte en inner join eller likekobling, så vil de ikke det, siden kunde_id fra kunde ikke kan matches med noen kunde_id i ordre. Da må man i dette tilfelle skrive om til en left outer join. Dersom man i forrige oppgave brukte en likekobling eller inner join, så må man bruke en left outer join her (eventuelt right outer join hvis rekkefølgen på tabellene var omvendt).

j) En rad i tabllen Kunde består av serial (integer: 4 byte + navn: 50*2 byte (antar 2 byte per tegn for unicode) + epost: 50+*2 byte = 104 byte. En rad i tabellen Ordre beatår av ordre_id: serial (integer) 4 byte + kunde_id: integer 4 bye = 8 byte. Totalt får vi da: 104*100 + 8*1000 = 10400 + 8000 = 18400 byte = 18,4 kB. Disse beregningene avhenger selvsagt av hva man valgte for datatyper i disse tabellene.

Oppgave 4 (25% ~ 1 time) – NoSQL

4.1 XML og JSON

I «vedlegg 4.1» er det definert et XML Schema dokument (fil: ***kunde-ordre.xsd***) som spesifiserer hvordan en applikasjon ønsker å representere og utveksle informasjon om ordrer (f.eks. i en nettbutikk løsning).

Tips!

- *Default verdien for "minOccurs" og "maxOccurs" i en XML Schema definisjon er 1.*
- *xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="?"*

a) (3% ~ 7 min) Hva betyr det at et XML dokument er "velformet" og "gyldig"?

b) (6% ~ 15 min) Lag et eksempel på et velformet og gyldig XML dokument som referer til (kobles) og følger XML Schema dokumentet "kunde-ordre.xsd" (se vedlegg 4.1). Eksemplet skal inneholde en ordre med to ulike varer, der den ene varen oppgis med en "RabattKode".

c) (3% ~ 7 min) Skriv ditt eksempel fra oppgave b) om til et JSON dokument. Bruk element- og attributtnavn som nøkkelnavn (der det er behov for dette).

Forslag til løsning:

a) **Max 10 p**oeng

(3p) At et XML dokument er **velformet** betyr at det er syntaktisk korrekt, og dermed følger følgende regler:

- (1 p) Hvert XML-dokument har **ett og bare ett rot-element**
- (1 p) Alle elementer har både **start-tagg og slutt-tagg (eventuelt tomt element)**
- (1p) Hvert element kan ha **et antall attributter (ikke to attributter med samme navn)**
- (1p) **Kan ha nøstede elementer** som i HTML, men elementer **kan ikke overlappe**

(1p) At et XML dokument er **gyldig** betyr at det – I tillegg til å være velformet – (2p) tilfredsstiller en DTD og/eller en XML Schema definisjon.

a) **Max 10 poeng**

XML dokument som er gyldig mht. til XML Schema i vedlegg 4.1

NB! I XML schema gjengitt i eksamensoppgaven burde attributtet «VareId» vært definert med typen «VareIdType». I sensuren så har vi godtatt begge tolkninger, dvs. type «xs:string» (som definert i oppgaveteksen) eller «VareIdType» (som det egentlig er lagt opp til).

```
<xs:attribute type="xs:string" name="VareId" use="required" />
```

Løsningsforslag (vi benytter «VareIdType» for attributtet «VareId»):

```
<?xml version="1.0" encoding="utf-8" ?>
<Ordrer xmlns="http://eksamen.hvl.no/databaser"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://eksamen.hvl.no/databaser kunde-ordre2.xsd">
   <Ordre OrdreId="10012">
      <Kunde KundeId="1001">
         <KundeNavn>Ola Nordmann</KundeNavn>
         <Epost>ola.nordmann@gmail.com</Epost>
         <PostAdresse>
            <Adresse>Enebakkveien 12</Adresse>
            <Postnr>1205</Postnr>
            <Poststed>Enebakk</Poststed>
         </PostAdresse>
      </Kunde>
       <Varer>
         <Vare VareId="100201">
            <VareNavn>Helsport Fjellheimen 3 Pro</VareNavn>
            <Antall>1</Antall>
            <Pris>9000</Pris>
            <RabattKode>VIP</RabattKode>
         </Vare>
         <Vare VareId="100202">
            <VareNavn>JetBoil Mini</VareNavn>
            <Antall>1</Antall>
            <Pris>1200</Pris>
         </Vare>
      </Varer>
   </Ordre>
</Ordrer>
```

Poeng: (Max 10 p)

1 p – XML deklarasjon

1 p – Riktig rotelement <Ordrer>

1 p – Riktig default namespace (samme som targetNamespace i XML Schema)

1 p – Riktig mapping til XML schema

1 p – Riktig bruk av start og slutt tagger

2 p – Riktig nøsting av elementer («Ordrer» elementet består av en eller flere «Ordre» element. Et «Ordre» element består av et «Kunde element og et «Varer» element. «Varer» element består av et eller flere «Vare» element.)

1 p – Riktig bruk av attributter og elementer

1 p – Riktig verdi for KundeId og VareId (med henhold til restriksjon angitt i XML Schema)

1 p – Riktig verdi for RabattKode (med henhold restriksjon angitt i XML Schema)

b) **Max 10 poeng** (JSON dokument som representerer samme data som fra oppgave b):

```
{
  "Ordrer": [
    {
      "OrdreId": 10012,
      "Kunde": {
        "KundeId": 1001,
        "KundeNavn": "Ola Nordmann",
```

```
      "Epost": "ola.nordmann@gmail.com",
      "PostAdresse": {
        "Adresse": "Enebakkveien 12",
        "Postnr": "1205",
        "Poststed": "Enebakk"
      }
    },
    "Varer": [
      {
        "VareId": 100201,
        "VareNavn": "Helsport Fjellheimen 3 Pro",
        "Antall": 1,
        "Pris": 9000,
        "RabattKode": "VIP"
      },
      {
        "VareId": 100202,
        "VareNavn": "JetBoil Mini",
        "Antall": 1,
        "Pris": 1200
      }
    ]
  }
]
}
```


Poeng (Max 10 poeng):

2 p – Riktig bruk av hoved objekt «ordrer». PS! Her godtar vi også at man går rett på Array

2 p – Riktig bruk av objekter

2 p – Riktig bruk av nøkkel-verdi par

2 p – Riktig bruk av arrays

2 p – Ingen øvrige syntaks feil i JSON dokument og dokumentet fornuftig bygd opp

4.2 NoSQL

Når vi snakker om NoSQL databaser så er disse ofte distribuerte systemer, som utgjør et database kluster. Disse databasene bruker ofte teknikker som "Sharding" og "Replikering".

a) (2,5% ~ 6 min) Forklar kort hva som menes med "Sharding" når vi snakker om distribuerte database systemer? Hvilke fordeler gir denne typen partisjonering?
b) (2,5% ~ 6 min) Forklar kort hva som menes med "Replikering"? Hvilke fordeler og ulemper gir replikering?

MongoDB er et eksempel på en dokumentbasert NoSQL database. I henhold til CAP-teoremet (C=Consistency, A=Availability, P=Partition Tolerance) sier vi at MongoDB har et "CP" design.

c) (6% ~ 15 min) Forklar kort hva som ligger i begrepene "Consistency" (no: kapasitet), "Availability" (no: Tilgjengelighet) og "Partition Tolerance" (no: Partisjons toleranse), samt forklar kort CAP-teoremet?
d) (2% ~ 4 min) I tabellen under er det listet opp en del database systemer (til venstre), og en del egenskaper, spørringer og/eller fakta (til høyre). Du skal knytte sammen de ulike database systemene med egenskaper, spørringer, og/eller fakta.
   Svar ved å angi bokstav for database system, og list tall fra høyre kolonne.
   PS! Tallene på høyre side KAN matche/passe til flere av database systemene i venstre kolonne.

Eksempel (på svar metode):

A: (liste med tall fra høyre kolonne)
B: (liste med tall fra høyre kolonne)
osv.

Tips*! 0,5 poeng for hver riktig kobling, men -0,5 poeng ved feil. Total score vil uansett ikke bli mindre enn 0, men det kan lønne seg å ikke "gamble" hvis man ikke vet svaret.*

Database Systemer	Egenskaper/Fakta/Spørring
A. MongoDB	1) Er en nøkkel-verdi database
B. Cassandra	2) Er en dokumentbasert database
C. Neo4J	3) Er en kolonnedatabase
D. Amazon DynamoDB	4) Er en grafdatabase
E. PostgreSQL	5) Er en relasjonsdatabase
	6) Er et eksempel på et "CA"-design (CAP-teorem)

	7) Er et eksempel på et “CP”-design (CAP-teorem) 8) Er et eksempel på et “AP”-design (CAP-teorem) 9) Bruker spørrespråket “CQL” 10) Bruker spørrespråket “Cypher” 11) Bruker et propritært API basert på JSON 12) Bruker spørrespråket “SQL” 13) db.kurs.find({ semester: {$eq: "H"}} 14) MATCH (actor {name: "Tom Hanks"}) RETURN actor; 15) Kun “serverless”

Forslag til løsning:

a) **Max 10 poeng**

(1p) "Sharding" er en teknikk innen distribuerte database systemer hvor datasettet deles inn i mindre deler kalt "shards" (NB! Ikke nødvendig å vite at disse kalles "shards").

(3p) På denne måten partisjoneres data over flere datamaskiner eller servere.

(2p) Skalerbarhet: Dette gjør det mulig å håndtere store datamengder mer effektivt og skalere systemet horisontalt (billigere skalering, samt større muligheter til å skalere).

(2p) Kostnadseffektivitet: skalere systemet horisontalt (billigere skalering)

(2p) Bedre ytelse. Spørringer kan distribueres og prosesseres parallelt på flere noder. Sharding bidrar til å forbedre ytelse og tilgjengelighet i store distribuerte systemer ved å redusere belastningen på enkeltkomponenter. Data kan plasseres/ lokaliseres i "nærheten" der de brukes (f.eks. samme område, land og/eller verdensdel)

b) **Max 10 poeng**

(3p) "Replikering" i distribuerte database løsninger refererer til prosessen med å opprettholde identiske kopier av data på flere servere/ noder.

Fordeler med replikering inkluderer:

(1,5p) Økt tilgjengelighet: Hvis en server eller node mislykkes, kan spørringer og forespørsler bli rutet til en av de replikerte kopiene, slik at tjenesten forblir tilgjengelig.

(2p) Bedre ytelse: Ved å distribuere belastningen på flere kopier av data, kan replikering redusere flaskehalser og forbedre spørringsytelsen ved å tillate parallell spørringsbehandling.

(1,5p) Feiltoleranse/ data redundans: Replikering gjør det mulig å gjenopprette data fra en av de replikerte kopiene i tilfelle datakorruptering eller feil, og dermed øker systemets feiltoleranse / data redundans.

Ulemper med replikering inkluderer:

(2p) Økt kompleksitet. Synkroniseringsproblemer: Komplekst å synkronisere over replikaset og dermed holde data kopier **konsistente**.

Andre ulemper (som vi godtar):
(1p) Tar mye diskplass/kost (dobbel/trippel lagring av data)

(1p) Replikering over geografiske områder (nærhet til data)

c) Max 10 poeng.

CAP-teoremet er en prinsipiell modell for distribuerte systemer som beskriver tre viktige egenskaper: konsistens (Consistency), tilgjengelighet (Availability), og partisjonstoleranse (Partition Tolerance).

(2p) **Konsistens (Consistency)**: Dette refererer til at alle klienter ser den samme datavisningen til enhver tid, uavhengig av hvilken node de kobler seg til i systemet.

(Med andre ord, hvis en endring blir utført i systemet, vil alle etterfølgende spørringer reflektere denne endringen.)

(2p) **Tilgjengelighet (Availability)**: Dette betyr at systemet alltid svarer på spørringer, selv om enkelte deler av systemet kan være nede. Tilgjengelighet handler om å sikre at systemet forblir operativt og kan behandle forespørsler, selv i tilfelle feil eller nedetid.

(2p) **Partisjonstoleranse (Partition Tolerance):** Dette refererer til systemets evne til å fortsette å fungere selv om det oppstår nettverksfeil som fører til at meldinger ikke kan leveres mellom noen noder. Med andre ord, systemet kan operere selv når det oppstår partisjoner i nettverket.

(3p) CAP-teoremet postulerer at et distribuert system ikke kan oppfylle alle tre egenskapene (konsistens, tilgjengelighet og partisjonstoleranse) samtidig. I stedet kan systemet bare oppnå to av disse egenskapene samtidig.

(1p) Distribuerte systemer kan dermed kategoriseres etter følgende design (nok å vite kategoriene her):

- **CP-design**: Hvis man prioriterer konsistens og partisjonstoleranse (CP), vil systemet "garantere"/ prøve/ prioritere å oppnå at alle klienter ser samme og konsistente data, selv i tilfelle partisjoner i nettverket, men dette kan gå på bekostning av tilgjengeligheten hvis noen noder mislykkes.
- **AP-design**: Hvis man prioriterer tilgjengelighet og partisjonstoleranse (AP), kan systemet alltid svare på spørringer og være tilgjengelig, selv i tilfelle partisjoner i nettverket, men dette kan gå på bekostning av konsistens hvis dataene på forskjellige noder er inkonsistente.

- **CA-design**: Når data lagres sentralt, som ved relasjonsdatabaser, så unngår man muligheten for partisjoner (og dermed partisjons toleranse). Neo4J er også en slik type database.

Ekstrapoeng (1p):
CAP-teoremet hjelper utviklere og arkitekter med å ta informerte beslutninger om design av distribuerte systemer, slik at de kan velge de egenskapene som er viktigst for deres spesifikke brukstilfeller.

d) Maks 10 poeng (0,7 poeng per riktige).

NB! For å få poengsum til å passe med en maximum på 10, så har jeg endret poeng per riktig til 0,7 (i stedet for 0,5 som det står i oppgaven).

A: 2, 7, 13, (godtar også 11)
B: 3, 8, 9
C: 4, 6, 10, 14 (godtar også 7,8)
D: 1, 8, 11, 15
E: 5, 6, 12

Vedlegg 4.1: (Filnavn: kunde-ordre.xsd)

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified"
  xmlns="http://eksamen.hvl.no/databaser"
  targetNamespace="http://eksamen.hvl.no/databaser"
  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="Ordrer">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="Ordre" type="OrdreType" minOccurs="1"
        maxOccurs="unbounded" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:complexType name="OrdreType">
    <xs:sequence>
      <xs:element name="Kunde" type="KundeType"/>
      <xs:element name="Varer" type="VarerType"/>
    </xs:sequence>
    <xs:attribute name="OrdreId" type="xs:integer" use="required"/>
  </xs:complexType>
  <xs:complexType name="KundeType">
    <xs:sequence>
      <xs:element name="KundeNavn" type="xs:string" />
      <xs:element name="Epost" type="xs:string" />
      <xs:element type="PostAdresseType" name="PostAdresse" />
    </xs:sequence>
    <xs:attribute name="KundeId" type="KundeIdType" use="required" />
  </xs:complexType>
  <xs:complexType name="PostAdresseType">
    <xs:sequence>
      <xs:element name="Adresse" type="xs:string"/>
      <xs:element name="Postnr" type="xs:string"/>
      <xs:element name="Poststed" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
```

```
  <xs:simpleType name="VareIdType">
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="100000" />
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="KundeIdType">
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="1000" />
      <xs:maxExclusive value="100000" />
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="VarerType">
     <xs:sequence>
        <xs:element type="VareType" name="Vare" minOccurs="1" maxOccurs="unbounded"
        />
     </xs:sequence>
  </xs:complexType>
  <xs:complexType name="VareType">
    <xs:sequence>
      <xs:element name="VareNavn" type="xs:string"/>
      <xs:element name="Antall" type="xs:integer"/>
      <xs:element name="Pris" type="xs:decimal"/>
      <xs:element name="RabattKode" minOccurs="0" maxOccurs="1">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="VIP" />
            <xs:enumeration value="KAMPANJE20" />
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
    </xs:sequence>
   <xs:attribute type="xs:string" name="VareId" use="required" />
  </xs:complexType>
</xs:schema>
```